Detecting Speed and Tempo Alterations in Speech Recordings

acp25ck

University of Sheffield

Abstract

This report investigates Scikit-learn classifiers for detecting speed modification and tempo modification in speech recordings using the provided FBANK features. Two baseline models are being provided as a base for improvement and comparison. The features are flattened by averaging each FBANK frequency channel over time, and the resulting 64-dimensional vector is classified using k-nearest neighbours (kNN) [1]. As for speed modification, with feature standardisation and hyper-parameter tuning of k, the accuracy improved from 79.2% to 86.6% on the evaluation set. In contrast, tempo modification remains low (best 31.4%), even after testing multiple classifiers (kNN, Logistic Regression, linear SVM, and Random Forest).

1. Introduction

Audio recordings can be modified by changing *speed* (playback rate, which affects both pitch and duration) or *tempo* (time-stretching that changes duration while largely preserving pitch). The aim of this assignment is to classify short segments of speech recordings into different categories for speed and tempo using pre-computed filter bank features derived from CMU Arctic recordings [2].

This report describes the baseline system, the improvements compared with the baseline model for speed and tempo classification, and an analysis of why some approaches generalise well for speed but struggle for tempo when temporal information is collapsed.

2. System Description

Each training/evaluation file is a `joblib` dictionary containing `features` and `target`. The FBANK features are flattened vectors derived from 64 Mel filterbank channels across time frames. Let $X \in \mathbb{R}^{N \times D}$ denote the flattened FBANK matrix for N samples.

Temporal pooling. For both tasks, each feature vector is reshaped into $(64, T)$ and reduced by taking the per-channel temporal mean, producing a 64-dimensional vector:

$$\phi(x) = \frac{1}{T} \sum_{t=1}^{T} x_{:,t}. \tag{1}$$

This pooling makes the representation fixed-length and lightweight, but discards temporal structure.

Classifiers. For *speed*, the final system uses `StandardScaler` followed by kNN with tuned neighbourhood size k. For *tempo*, several classifiers were tested on the same pooled representation, including kNN, Logistic Regression, linear SVM, and Random Forest.

kNN uses Euclidean distance (default settings in scikit-learn). The tuned hyperparameter for speed is the number of neighbours k. For tempo, the same pooled representation $\phi(x)$ is used, and 4 classifiers are evaluated (kNN, Logistic Regression, linear SVM, and Random Forest). Unless stated, classifier hyperparameters use scikit-learn defaults.

3. Experiments

I first reproduced the provided baseline system for both tasks using the supplied training split (`*.train.joblib`) and evaluated performance on the supplied evaluation split (`*.test1.joblib`). All experiments were implemented as scikit-learn pipelines to ensure consistent application of preprocessing steps during training and evaluation.

Speed task. Starting from the baseline (temporal mean pooling + kNN with $k = 1$), I tested feature standardisation using `StandardScaler` and then tuned the kNN hyper-parameter $k \in \{1, 3, 5, 7\}$. Only one component was changed at a time to isolate the effect of each modification.

Tempo task. The same pooled FBANK is used. I first repeated the tuning I did with kNN hyper-parameter as the speed class, and compared its performance with and without `StandardScaler`. I then evaluated alternative classifiers (Logistic Regression, linear SVM, and Random Forest) to test whether a different classifier could achieve better performance on the tempo classes. Classifier parameters were set to scikit-learn defaults. For Random Forest, I used a reasonable number of trees without performance trade-offs and a limited depth to control model size and avoid excessive complexity.

Model size constraint. Trained models were saved using `joblib` and kept below the 80 MB size limit enforced by the provided evaluation script. Final performance results are reported in the Results section.

4. Results and Analysis

Figure 1 provides a representation of FBANK with slow and fast modifications. For speed modification, changes in pitch and duration produce visible differences in the spectral structure. In contrast, tempo modification mainly changes timing while largely preserving pitch, so temporally averaged FBANK features are expected to be worse at discrimination. This observation is consistent with the results below.

Table 1 summarises the accuracy of speed classification. The final speed system (StandardScaler + kNN with $k = 7$) achieves **86.6%**.

Table 1: *Speed classification accuracy for different kNN configurations.*

Configuration	% Correct
Baseline (kNN, $k = 1$)	79.2
+ StandardScaler	81.0
kNN ($k = 3$)	82.4
kNN ($k = 5$)	84.0
kNN ($k = 7$)	**86.6**

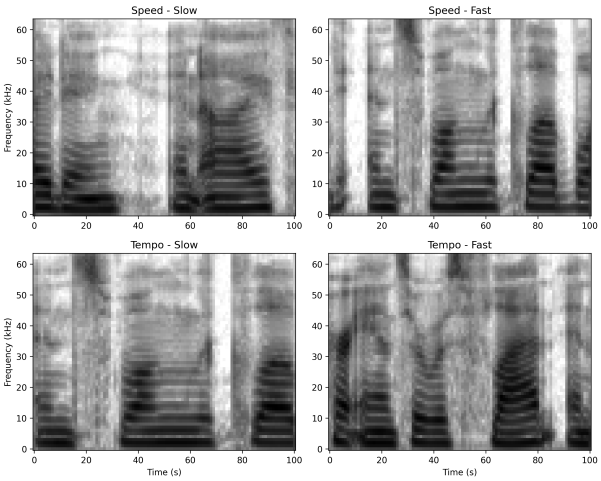

Figure 1: *Example FBANK representations of slow and fast* speed *and* tempo *modifications.*

For speed detection, standardisation improves kNN performance, and increasing k consistently increases accuracy. Most errors are expected to occur mainly between neighbouring classes (e.g., normal vs. fast) because speed is continuous, but during classification it is categorised into discrete categories.

Table 2 summarises tempo classification accuracy across multiple models. The final tempo system (Random Forest) achieves **31.4%**.

Table 2: *Tempo classification accuracy for different models and configurations.*

Model / Configuration	% Correct
Baseline (kNN, $k = 1$)	24.6
kNN + StandardScaler	24.2
kNN ($k = 3$)	24.6
kNN ($k = 5$)	24.8
kNN ($k = 7$)	24.8
kNN ($k = 5$, no scaling)	26.8
Logistic Regression	27.8
Linear SVM	30.0
Random Forest	**31.4**

Across all the tested models, the accuracy of tempo is relatively low. This is consistent with the chosen representation: averaging over time frames removes the structure that distinguishes tempo changes, while only the mainly spectral content is preserved, which changes relatively little under tempo modification. The Random Forest achieves the best result (31.4%), suggesting some non-linear separability in the pooled features. However, the overall performance indicates that achieving higher accuracy requires preserving the temporal structure. In practice, many errors are likely to arise because clips with tempo up/down can share similar pooled spectral statistics.

5. Discussion and Conclusions

The experiments show that with the final speed system (StandardScaler + kNN with $k = 7$), significant improvements in performance, with 86.6% accuracy on the evaluation set, and show clear sensitivity to hyper-parameter tuning: increasing k consistently improves performance relative to the baseline. In contrast, the results of tempo classification remain poor across all tested models; even with the best system (Random Forest), only 31.4% accuracy is achieved. The issue is that the current feature pipeline uses temporal mean pooling, which removes frame timing information essential for distinguishing tempo changes.

During this experiment, each training sample duration was fixed by the provided dataset. Testing cannot be done using longer/shorter audio to evaluate whether accuracy will be affected. If there were more time, the next step could be experimenting with averaging the whole recording clip for pooling, making it more features to work with, adding variance with mean, or using the full FBANK (keeping it as 64 x T) as input so the model can evaluate with more complete time information, to better capture tempo modification and improve the performance of the model.

6. References

[1] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[2] J. Kominek and A. W. Black, "The cmu arctic speech databases," in *Ssw*, 2004.